K.R. MANGALAM UNIVERSITY

SCHOOL OF ENGINEERING &TECHNOLOGY

Department of Computer Science & Technology

Assignment Number: 2

Unit Title: Linear Data Structures

Assignment Title: Linear Data Structures

Course Code: ETCCDS202

Course Name: Data Structures

Faculty: Mrs. Neetu Chauhan

Student Name: Jiya Gupta

Roll Number: 2501730209

Section: A

Programme: B. Tech CSE (Specialization in AI & ML)

1. Introduction

This project implements important linear data structures in Python from scratch. The data structures included are:

- Dynamic Array
- Singly Linked List
- Doubly Linked List
- Stack using Singly Linked List
- Queue using Singly Linked List

A real application, **Balanced Parentheses Checker**, is also implemented using Stack.

2. Task 1: Dynamic Array Simulation

Implementation

A class `DynamicArray` was created with:

- `capacity` → total size of array
- `size` → number of elements stored
- `append(x)` → adds element at end
- `pop()` → removes last element
- automatic resizing when full

When the array becomes full, a new array of **double capacity** is created and old elements are copied.

Amortized Time Complexity of append()

- Normal append inserts directly at end → **O(1)**
- When array is full, resizing and copying all elements is needed → **O(n)**

But resizing does not happen every time.

Conclusion

Most append operations are O(1). Some are O(n) during resize. Overall average cost of append is:

Amortized Complexity = O(1)

3. Task 2: Linked List Operations

(A) Singly Linked List

Each node stores:

- data
- next pointer

Operations Implemented

- insert_at_beginning(x)
- insert_at_end(x)
- delete_by_value(x)
- traverse()

(B) Doubly Linked List

Each node stores:

- data
- next pointer
- previous pointer

Operations Implemented

- insert_after_node(target,x)
- delete_at_position(pos)

4. Task 3: Stack and Queue Using Singly Linked List

(A) Stack ADT

Stack follows:

LIFO (Last In First Out)

Operations:

- push(x)
- pop()
- peek()

Push and pop are performed at head for efficiency.

Time Complexity

Operation	Complexity
Push	O(1)
Pop	O(1)
Peek	O(1)

(B) Queue ADT

Queue follows:

FIFO (First In First Out)

Operations:

- enqueue(x)
- dequeue()
- front()

Head and tail pointers are maintained.

Time Complexity

Operation	Complexity
Enqueue	O(1)
Dequeue	O(1)
Front	O(1)

5. Task 4: Balanced Parentheses Checker

A function is_balanced(expr) was implemented using the created Stack.

Supported brackets:

- ()
- {}
- []

Logic

- Push opening brackets into stack
- On closing bracket, pop and match pair
- If all brackets match and stack becomes empty → Balanced

Test Cases

Input	Output
([])	True
([)]	False
(((	False
""	True

6. Learning Outcomes

After completing this assignment, I learned:

- Dynamic array resizing concept
- Difference between arrays and linked lists
- Stack and Queue implementation using linked list
- Importance of time complexity
- Real use of Stack in expression checking

7. Conclusion

This assignment helped in understanding linear data structures practically. Implementing each structure from scratch improved coding skills and understanding of internal working of arrays, linked lists, stacks, and queues.

8. References

- Python Documentation
- Class Notes
- Data Structures Study Material